

1. Reusing Context without Repaying for It

Prompt caching reduces the cost of repeatedly processing identical or stable prefix context in supported API workflows. The engineering implication is that stable instructions and reference material should be structured so they can be reused rather than rewritten unpredictably on every turn.

Caching is a performance mechanism, not an excuse to carry irrelevant context. Cheap noise can still harm attention and behavior.

2. Stable Prefix, Changing Task

Imagine an evaluation harness that repeatedly supplies the same repository conventions, tool definitions, and long reference document while changing only the task at the end. A stable prefix can benefit from caching. If every request regenerates the same content in a different order or wording, reuse becomes harder.

This suggests a design principle: separate durable context from task-specific variation when the interface allows it.

3. Astra's Large Context Window

The GPT-6 Astra API model page lists a 1,050,000-token context window and 128,000 maximum output tokens. That capacity changes what is possible, but not the value of selection. OpenAI also documents different pricing behavior for prompts above a large-input threshold. Large context is therefore both a capability and a resource decision.

A million-token window should not become a million-token default.

4. Configuration Updates

Current Astra guidance supports changing reasoning effort during an ongoing supported API conversation through configuration updates while preserving cache benefits. The conceptual advantage is phase-specific compute. Exploration can run at lower effort, a difficult design decision can escalate, and routine application of the decided pattern can return to a cheaper setting.

The hard part is deciding when a phase actually changed.

5. An Effort Schedule

A disciplined long task might use:

Reconnaissance: Low or Medium, because tools and search provide much of the information.

Decision point: High or above when alternatives require difficult tradeoffs.

Mechanical implementation: Medium if the design is now explicit.

Adversarial review: High when the goal is to find subtle counterexamples.

Routine cleanup: Low.

This is an example, not a universal schedule. A difficult coding implementation may itself need high effort. The principle is to spend reasoning where uncertainty and consequence are concentrated.

6. Cache versus Notes

Caching preserves computational reuse for stable input. Notes preserve semantic state across long work.

Searchable earlier context preserves recoverability. These mechanisms should not be conflated.

Cache answers: can repeated input be processed more efficiently?

Notes answer: what state should remain salient?

Search answers: can an older detail be recovered?

Durable files answer: what should future sessions and humans know independent of this thread?

7. Measuring the Right Thing

Optimization claims require an evaluation. If changing effort reduces tokens but increases human corrections, the apparent savings may be false. If caching lowers API cost but the workflow becomes harder to maintain, the total cost may still rise. Measure task completion, corrections, latency, and resource use together.

References: OpenAI, GPT-6 Astra model guidance; OpenAI, GPT-6 Astra model reference; OpenAI, prompt caching documentation.

8. Large-Prompt Price Boundary

The current Astra model page documents a price multiplier for prompts above 272K input tokens. That creates a discontinuity: adding context near the threshold can increase the price of the full request, not merely the marginal extra tokens. A workflow that routinely operates near the boundary should measure actual input sizes and decide whether summarizing, retrieving selectively, or splitting work changes total task cost without harming success.

9. Prompt Shape and Cacheability

Two prompts can carry the same facts but have different reuse. If stable policies, schemas, and tools remain in the same prefix while the task-specific request comes later, repeated calls can reuse more cached material. If stable material is regenerated with timestamps, reordered lists, or small wording changes every turn, cache hits may fall. The optimization should never distort instruction priority just to preserve a cache; correctness remains the first constraint.

10. Phase Boundary Test

Change reasoning effort only at a meaningful boundary. Ask: did the kind of uncertainty change? Reconnaissance uncertainty is often about missing facts. Architecture uncertainty is about competing plans. Implementation uncertainty is about translating a decided plan correctly. Review uncertainty is about hidden failure modes. These phases justify different tools and sometimes different effort. Changing settings without a new information problem is tuning noise.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 15: Cache and Reasoning Changes

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>